

RAPPORT

Projet CI/CD — EC06

Mise en place d'un système d'automatisation d'intégration et de déploiement continu

Scénario professionnel : Catal-Log

Formation ASRC — RNCP39611 — Bloc BC02

Auteur : Étudiant11

Dépôt : <https://github.com/Etudiant11-hub/projet-cicd-ec06>

Date : Juillet 2026

SOMMAIRE

- 1. Introduction 3
- 2. Objectifs du projet 3
- 3. Architecture technique mise en place..... 4
 - 3.1 Structure du dépôt 4
 - 3.2 Conteneurisation (Dockerfile) 5
 - 3.3 Orchestration légère (compose.yml) 5
- 4. Schéma de la chaîne CI/CD 6
- 5. Détail des workflows GitHub Actions 6
 - 5.1 01-ci.yml — Intégration continue..... 6
 - 5.2 02-publish-ghcr.yml — Publication dans GHCR..... 7
 - 5.3 03-promote.yml — Validation et promotion 7
- 6. Tests réalisés 9
 - 6.1 Test automatisé dans GitHub Actions..... 9
 - 6.2 Test local avec Docker..... 9
 - 6.3 Test avec Docker Compose 9
 - 6.4 Simulation de mise à l'échelle (scaling)10
- 7. Preuves obtenues.....11
- 8. Sécurité minimale mise en œuvre12
 - 8.1 Permissions des workflows.....12
 - 8.2 Gestion des secrets.....12
 - 8.3 Rollback.....12
 - 8.4 Sauvegarde et restauration12
 - 8.5 Éléments complémentaires12
- 9. Difficultés rencontrées et apprentissages.....13
- 10. Conclusion.....14

1. Introduction

Je présente dans ce rapport la chaîne CI/CD que j'ai mise en place pour l'évaluation EC06, dans le cadre du scénario professionnel Catal-Log. L'objectif de ce projet était de construire, tester, publier et promouvoir automatiquement une image Docker Nginx contenant un site web statique, en utilisant GitHub, GitHub Actions et GitHub Container Registry (GHCR), avec une simulation des environnements de recette et de production.

Je détaille ici l'ensemble des choix techniques que j'ai faits, le fonctionnement de chaque brique de la chaîne, les tests que j'ai réalisés (en local et automatisés), les preuves obtenues à chaque étape, ainsi que mon analyse de ce qu'il faudrait ajouter pour transformer cette simulation en une véritable mise en production.

2. Objectifs du projet

Les objectifs que je devais atteindre étaient les suivants :

- Versionner mon projet dans un dépôt GitHub individuel.
- Mettre en place un contrôle automatique du code à chaque modification.
- Construire une image Docker à partir d'un Dockerfile.
- Tester automatiquement le conteneur dans GitHub Actions.
- Publier l'image dans GitHub Container Registry (GHCR).
- Identifier chaque image avec un tag et un digest uniques.
- Valider l'image dans un environnement de recette simulé.
- Promouvoir manuellement le même artefact vers un environnement de production simulé, sans reconstruction.
- Documenter mes choix techniques, mes limites et mes preuves de manière vérifiable.

3. Architecture technique mise en place

3.1 Structure du dépôt

J'ai organisé mon dépôt de la façon suivante, conformément aux livrables attendus :

```
|-- .github
|   |-- workflows
|   |   |-- 01-ci.yml
|   |   |-- 02-publish-ghcr.yml
|   |   |-- 03-promote.yml
|-- Dockerfile
|-- README.md
|-- compose.yml
|-- docs
|   |-- 01-cadrage-projet.md
|   |-- 02-schema-chaine-cicd.md
|   |-- 03-fiche-tests.md
|   |-- 04-preuve-image.md
|   |-- 05-preuve-recette.md
|   |-- 06-preuve-promotion.md
|   |-- 07-securite-minimale.md
|   |-- 08-compte-rendu-final.md
|-- site
|   |-- index.html
|   |-- version.json
```

3.2 Conteneurisation (Dockerfile)

Pour la conteneurisation, je suis parti de l'image officielle Nginx :1.27-alpine, choisie pour sa légèreté et sa maintenance régulière. Mon Dockerfile copie le contenu de mon dossier site/ dans le répertoire servi par Nginx, ajuste les permissions, expose le port 80 et déclare un HEALTHCHECK qui vérifie périodiquement que le serveur répond correctement. J'ai également ajouté des labels OCI pour documenter l'image (titre, description, dépôt source).

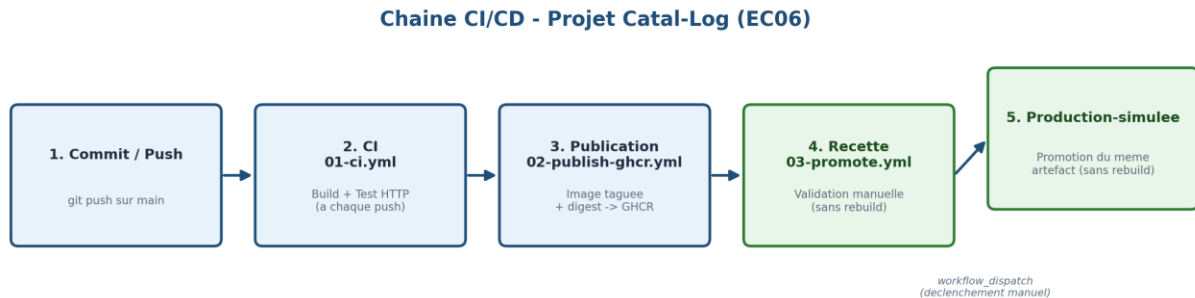
3.3 Orchestration légère (compose.yml)

J'ai écrit un fichier compose.yml qui décrit deux services : le service web, construit depuis mon Dockerfile local, et un second service tester, basé sur l'image curlimages/curl, qui dépend du service web et exécute automatiquement des requêtes HTTP de vérification dès que celui-ci est démarré. Ce second service me permet de démontrer une coordination simple entre plusieurs conteneurs, ce qui répond à l'exigence de la compétence C13 sur l'orchestration légère.

Je suis conscient que Docker Compose ne remplace en rien un orchestrateur de production comme Kubernetes : il ne gère ni la répartition de charge, ni la haute disponibilité, ni le déploiement progressif. J'explique ce point plus en détail dans la partie consacrée au scaling.

4. Schéma de la chaîne CI/CD

Le schéma ci-dessous représente le déroulement complet de ma chaîne, du commit jusqu'à la promotion en production simulée.



Environnements GitHub simulés :

- recette : validation avant promotion
- production-simulée : cible finale, artefact promu sans reconstruction

Runners : GitHub-hosted (ubuntu-latest)

Registre : GitHub Container Registry (GHCR)

Auth : secrets.GITHUB_TOKEN (temporaire)

5. Détail des workflows GitHub Actions

Flux de travail	Événement	Statut	Bifurquer	Acteur
02 - Publication GHCR n° 5 : Commit 4661387 poussé par Etudiant11-hub	workflow_dispatch	Reussite		principal
01 - CI - Compilation et test n° 5 : Commit 4661387 poussé par Etudiant11-hub	push	Reussite		principal
02 - Publication GHCR n° 4 : Commit 8bc8d1 poussé par Etudiant11-hub	workflow_dispatch	Reussite		principal
01 - CI - Compilation et test n° 4 : Commit 8bc8d1 poussé par Etudiant11-hub	push	Reussite		principal
01 - CI - Compilation et test n° 3 : Commit 26c440 poussé par Etudiant11-hub	workflow_dispatch	Reussite		principal
02 - Publication GHCR n° 3 : Commit 26c440 poussé par Etudiant11-hub	push	Reussite		principal

5.1 01-ci.yml — Intégration continue

Ce workflow se déclenche à chaque push ou pull request, ainsi que manuellement. Je l'ai conçu pour qu'il vérifie d'abord la présence des fichiers attendus dans mon dépôt (Dockerfile, compose.yml, site/index.html, site/version.json, docs/08-compte-rendu-final.md), puis qu'il valide la syntaxe de mon compose.yml. Il construit

Événement	Statut	Bifurquer	Acteur
01 - CI - Compilation et test n° 5 : Commit 4661387 poussé par Etudiant11-hub	Reussite		principal
01 - CI - Compilation et test n° 4 : Commit 8bc8d1 poussé par Etudiant11-hub	Reussite		principal
01 - CI - Compilation et test n° 3 : Commit 26c440 poussé par Etudiant11-hub	Reussite		principal
01 - CI - Compilation et test n° 2 : Commit c6b070 poussé par Etudiant11-hub	Reussite		principal
01 - CI - Compilation et test n° 1 : Commit 4661387 poussé par Etudiant11-hub	Reussite		principal

ensuite mon image Docker, démarre un conteneur de test, et vérifie avec curl que la page d'accueil et le fichier version.json répondent correctement en HTTP. Ce workflow déclare uniquement la permission contents : read, car il ne fait que lire mon dépôt et exécuter des tests, sans jamais écrire ni publier quoi que ce soit.

5.2 02-publish-ghcr.yml — Publication dans GHCR

Ce workflow se déclenche uniquement lors d'un push sur la branche main. Il reconstruit mon image avec docker/build-push-action, puis la publie dans GitHub Container Registry avec deux tags : sha-<commit>, qui trace précisément le commit d'origine, et latest, qui pointe toujours vers la dernière version stable. L'authentification vers GHCR se fait via docker/login-action, en utilisant secrets.GITHUB_TOKEN, un jeton temporaire généré automatiquement par GitHub pour chaque exécution. Ce workflow nécessite la permission packages : write en plus de contents : read, car il doit pouvoir écrire dans le registre.

Actes Nouveau flux de travail

Tous les flux de travail

01 - CI - Construction et test

02 - Publication GHCR

03 - Promotion recette vers production...

Gestion

- Caches
- Déploiements
- Attestations
- Coueurs
- Métriques d'utilisation
- Indicateurs de performance

02 - Publication GHCR

02-publish-ghcr.yml

5 exécutions de flux de travail

Événement Statut Bifurquer Acteur

Ce flux de travail possède un déclencheur d'événement workflow_dispatch .

Exécuter le flux de travail

✓ Rédaction compte rendu final	02 - Publication GHCR n°5 : Commit 4661387 poussé par Etudiant11-hub	principal	Aujourd'hui à 09h11	...
✓ Mise à jour fiche tests avec résultats réels	02 - Publication GHCR n°4 : Commit f8c8d1 poussé par Etudiant11-hub	principal	Aujourd'hui à 09h06	...
✓ Ajout preuves recette et promotion	02 - Publication GHCR n°3 : Commit 2fcb4d poussé par Etudiant11-hub	principal	Aujourd'hui à 09h48	...
✓ Ajout preuve image GHCR	02 - Publication GHCR n°2 : Commit efd870 poussé par Etudiant11-hub	principal	Aujourd'hui à 09h21	...
✓ Initialisation du projet CICD EC06	02 - Publication GHCR n°1 : Commit 9f3e1df poussé par Etudiant11-hub	principal	Aujourd'hui à 09h06	...

5.3 03-promote.yml — Validation et promotion

Ce workflow ne se déclenche jamais automatiquement : je le lance manuellement via workflow_dispatch, en indiquant le tag de l'image que je souhaite promouvoir. Il comporte deux jobs successifs. Le premier, validate-recette, s'exécute dans l'environnement GitHub recette : il télécharge l'image déjà publiée avec un simple docker pull, sans jamais la reconstruire, la démarre, puis vérifie qu'elle répond correctement en HTTP. Le second job, promote-production-simulee, ne démarre que si le premier a réussi ; il s'exécute dans l'environnement production-simulee et réutilise exactement le même artefact téléchargé : il le retague en production-simulee

Actes Nouveau flux de travail

Tous les flux de travail

01 - CI - Construction et test

02 - Publication GHCR

03 - Promotion recette vers production...

Gestion

- Caches
- Déploiements
- Attestations
- Coueurs

03 - Promotion recette vers production-simulée

03-promote.yml

1 exécution du flux de travail

Événement Statut Bifurquer Acteur

Ce flux de travail possède un déclencheur d'événement workflow_dispatch .

Exécuter le flux de travail

✓ 03 - Promotion recette vers production-simulée	03 - Promotion recette vers production-simulée #1 : Exécutée manuellement par Etudiant11-hub	principal	Aujourd'hui à 09h30	...
--	--	-----------	---------------------	-----

avec docker tag, puis le republie avec docker push. À aucun moment ce workflow ne contient d'instruction docker build, ce qui garantit que l'image promue en production est rigoureusement identique à celle validée en recette.

6. Tests réalisés

6.1 Test automatisé dans GitHub Actions

Mon workflow 01-ci.yml s'est exécuté avec succès dès mon premier push : le build de l'image et les deux requêtes HTTP (page d'accueil et version.json) ont réussi. Ce test s'exécute dans un environnement identique et contrôlé à chaque exécution, ce qui garantit sa reproductibilité, contrairement à un test purement local.

6.2 Test local avec Docker

Comme Docker est installé et fonctionnel sur ma machine personnelle, j'ai pu réaliser un test local réel avant même de pousser mon code sur GitHub. J'ai construit mon image avec docker build, je l'ai démarrée avec docker run, puis j'ai vérifié avec curl que la page d'accueil et version.json répondaient correctement. J'ai obtenu les mêmes résultats positifs qu'en CI.

6.3 Test avec Docker Compose

J'ai ensuite lancé docker compose up --build afin de vérifier la coordination entre mes deux services. Le service web a démarré normalement, avec son HEALTHCHECK actif, et le service tester a exécuté ses propres requêtes curl vers le service web sur le réseau interne cidc_net. J'ai bien vu apparaître le contenu HTML et JSON dans les logs, et le conteneur tester s'est terminé avec le code de sortie 0, ce qui confirme la réussite du test d'intégration entre mes deux conteneurs.

```
tester-1 exited with code 0
tester-1 |         font-weight: bold;
tester-1 |         font-size: .85rem;
tester-1 |         letter-spacing: .03em;
tester-1 |     }
tester-1 |     ul { line-height: 1.7; }
tester-1 |     li::marker { color: #1f4e79; }
tester-1 |     footer {
tester-1 |         margin-top: 32px;
tester-1 |         padding-top: 16px;
tester-1 |         border-top: 1px solid #e8eef5;
tester-1 |         font-size: .9rem;
tester-1 |         color: #5d6d7e;
tester-1 |     }
tester-1 |     code {
tester-1 |         background: #f4f7fb;
tester-1 |         padding: 2px 6px;
tester-1 |         border-radius: 4px;
tester-1 |         font-size: .9em;
tester-1 |     }
tester-1 | </style>
tester-1 | </head>
tester-1 | <body>
tester-1 | <main>
tester-1 |     <p class="badge">Projet CIDC - EC06</p>
tester-1 |     <h1>Site statique Catal-Log</h1>
tester-1 |     <p>Cette page est servie par Nginx dans une image Docker construite, testée, publiée et promue avec GitHub Actions.</p>
tester-1 |     <ul>
tester-1 |         <li>Conteneurisation avec Dockerfile.</li>
tester-1 |         <li>Tests automatisés dans GitHub Actions.</li>
tester-1 |         <li>Publication dans GitHub Container Registry.</li>
tester-1 |         <li>Promotion simulée sans reconstruire l'image.</li>
tester-1 |     </ul>
tester-1 |     <footer>Réalisé par Etudiant11 &middot; Formation ASRC &middot; <code>RNCP39611BC02</code> &middot; EC06</footer>
tester-1 | </main>
tester-1 | </body>
tester-1 | </html>
tester-1 | {
tester-1 |     "projet": "Projet CIDC",
tester-1 |     "contexte": "EC06 - ASRC",
tester-1 |     "organisation": "Catal-Log",
tester-1 |     "auteur": "Etudiant11",
tester-1 |     "version": "1.0.0",
tester-1 |     "date": "2026-07-01"
tester-1 | }
web-1 | 127.0.0.1 - - [02/Jul/2026:10:17:07 +0000] "GET / HTTP/1.1" 200 1819 "-" "Wget" "-"
```

6.4 Simulation de mise à l'échelle (scaling)

J'ai simulé une mise à l'échelle avec la commande `docker compose up -d --scale web=2`. La commande `docker compose ps` m'a confirmé le démarrage de deux instances distinctes du service web, toutes deux à l'état Up, avec leur HEALTHCHECK actif, chacune exposant le port 80 en interne. Cela démontre que plusieurs instances identiques peuvent cohabiter sans conflit sur le même réseau.

Je précise toutefois les limites de cette simulation : il n'existe aucun répartiteur de charge devant ces deux instances, donc rien ne distribue réellement le trafic entre elles ; il n'y a pas de haute disponibilité réelle, puisque tout dépend de mon unique machine hôte ; et il n'y a pas de supervision centralisée (pas de monitoring ni d'alerting). Cette simulation reste donc un exercice pédagogique de démonstration, et ne remplace en aucun cas un orchestrateur de production tel que Kubernetes.

```
✓Network projet-cicd-ec06_cicd_net Created 0.0s
✓Container projet-cicd-ec06-web-2 Started 0.6s
✓Container projet-cicd-ec06-web-1 Started 0.4s
✓Container projet-cicd-ec06-tester-1 Started 0.7s
```

7. Preuves obtenues

Le tableau ci-dessous récapitule les preuves concrètes que j'ai obtenues à chaque étape de ma chaîne.

Élément	Valeur / Lien
Dépôt GitHub	https://github.com/Etudiant11-hub/projet-cicd-ec06
Run CI (01-ci.yml)	Réussi — build + tests HTTP OK dès le premier push
Run publication GHCR	https://github.com/Etudiant11-hub/projet-cicd-ec06/actions/runs/28544541141
Image publiée	ghcr.io/etudiant11-hub/projet-cicd-ec06
Tags	Latest, sha-df3e1df
Digest	sha256:4bd03ff07d2c153fd15847f43189350e0450d72c301fc6b024e74ef7ad3bbac5
Validation recette	Digest observé identique au digest publié — même artefact confirmé
Promotion production-simulee	Tag cible : production-simulee — aucune étape docker build dans le job

Le point que je considère le plus important dans ce tableau est l'identité stricte entre le digest observé en recette et le digest publié initialement : cela prouve, de façon vérifiable et non déclarative, que l'image que j'ai promue en production simulée est exactement celle que j'avais testée et validée, sans aucune reconstruction intermédiaire.

← 02 - Publication GHCR

✓ Initialisation du projet CICD EC06 #1

Relancer toutes les tâches

...

Summary

Tous les emplois

publish

Détails de l'exécution

Usage

Workflow file

Déclenché par une pression il y a 14 heures

Statut

Durée totale

Artefacts

Etudiant11-hub poussé -> df3e1df principal

Succès

24 secondes

1

02-publish-ghcr.yml

sur: appuyer

publier 19 ans

8. Sécurité minimale mise en œuvre

8.1 Permissions des workflows

J'ai appliqué le principe du moindre privilège à chacun de mes workflows. 01-ci.yml ne dispose que de contents : read, car il n'a besoin ni d'écrire dans mon dépôt ni de publier quoi que ce soit. 02-publish-ghcr.yml et 03-promote.yml disposent en plus de packages : write, strictement nécessaire pour publier ou promouvoir des images dans GHCR. Aucun de mes workflows ne dispose d'un droit d'écriture sur le contenu du dépôt.

8.2 Gestion des secrets

Je n'ai stocké aucun secret dans mon code : tout ce qui est versionné dans Git reste visible dans l'historique, même après suppression ultérieure, ce qui est particulièrement risqué sur un dépôt public. L'authentification vers GHCR repose uniquement sur secrets.GITHUB_TOKEN, un jeton temporaire généré automatiquement par GitHub à chaque exécution, dont la portée est limitée aux permissions déclarées dans chaque workflow et qui expire à la fin du run. En production réelle, j'ajouterais tout secret supplémentaire (identifiants externes, clés d'API, certificats) dans GitHub Secrets ou un coffre-fort dédié comme HashiCorp Vault.

8.3 Rollback

Comme chaque image que je publie est identifiée par un tag et un digest uniques et immuables, je peux revenir à une version antérieure en relançant simplement mon workflow de promotion avec le tag de la version stable que je souhaite restaurer. Cette repromotion ne nécessite jamais de reconstruction, ce qui garantit qu'un rollback consiste uniquement à republier un artefact déjà validé, sans risque d'introduire un changement de comportement inattendu.

8.4 Sauvegarde et restauration

En cas de perte de mon environnement, je devrais pouvoir restaurer : mon dépôt GitHub (code, Dockerfile, compose.yml, site), qui contient déjà mes workflows et ma documentation puisqu'ils sont versionnés avec le code ; les images publiées dans GHCR, qu'il serait prudent de répliquer vers un second registre en production réelle ; la configuration de mes environnements GitHub (recette, production-simulee), qui n'est pas versionnée et devrait être redocumentée si perdue ; et enfin l'ensemble de mes preuves d'exécution, que j'ai conservées dans ma documentation.

8.5 Éléments complémentaires

Au-delà des trois points obligatoires, j'ai choisi de développer deux éléments complémentaires. Le premier est le contrôle des vulnérabilités : en production réelle, j'ajouterais un scan automatique de mon image avec un outil comme Trivy avant toute publication, afin de détecter d'éventuelles failles connues dans l'image de base ou les paquets installés. Le second est la séparation stricte des environnements : mes environnements GitHub recette et production-simulee illustrent déjà ce principe à mon échelle, mais en production réelle cette séparation irait plus loin, avec des comptes, registres ou réseaux physiquement distincts.

Environnements

Nouvel environnement

Vous pouvez configurer des environnements à l'aide de règles de protection, de variables et de secrets. [En savoir plus sur la configuration des environnements.](#)

production-simulée



recette



9. Difficultés rencontrées et apprentissages

Globalement, j'ai bien suivi la logique de chaque étape de ce projet, sans blocage majeur, notamment parce que j'ai pris soin de tester systématiquement chaque brique en local avant de la pousser sur GitHub.

L'apprentissage que je retiens comme le plus important est l'intérêt de tester automatiquement avant de publier. Voir mon workflow 01-ci.yml vérifier automatiquement que mon image se construit et répond correctement en HTTP, avant même qu'elle ne soit publiée dans GHCR, m'a permis de comprendre concrètement l'intérêt d'un pipeline CI/CD : détecter un problème le plus tôt possible, avant qu'il n'atteigne un environnement partagé ou la production. Je retrouve ce même principe dans ma logique de promotion : je ne promeus jamais une image vers production-simulee sans qu'elle ait d'abord été validée en recette, ce qui réduit le risque de déployer une version défectueuse.

10. Conclusion

Ce projet m'a permis de mettre en œuvre, de bout en bout, une chaîne CI/CD complète et fonctionnelle : versionnement, intégration continue, conteneurisation, publication dans un registre d'images, et promotion contrôlée entre environnements simulés, le tout appuyé par une documentation et des preuves vérifiables à chaque étape. Je considère avoir répondu à l'ensemble des exigences du socle obligatoire de l'évaluation EC06, en ayant une compréhension claire de chaque choix technique effectué et de ses limites dans une optique de production réelle.